# Lab of Digital Design

## Exercise 1 - Submission Report

4-bit Ripple Carry Adder (Full Adder - Behavioral)

---

| | |
|---|---|
| **Course:** | Lab of Digital Design |
| **Exercise:** | Exercise 1 - 4-bit Ripple Carry Adder |
| **Academic year:** | A.Y. 2026/2027 |
| **Report scope:** | Grouping of the SystemVerilog files in the project folder, identification of the ones that match the exercise, and the final submission set. |

**Exercise 1 - Task statement:**
*Design a 4-bit ripple carry adder using a behavioral description for the full adder. Create a testbench and use ModelSim to verify that the design has the expected behavior.*

# 1. Folder Layout

All SystemVerilog files related to Exercise 1 live under the 'Exercise 1' folder, split into two subfolders:

- Exercise 1 / Submission -> Files to be handed in (3 files)
- Exercise 1 / Practice -> Warm-up files following the lecture examples (4 files)

Each subfolder is described below with the role of every file.

## Exercise 1 / Practice

These files reproduce the lecture flow (Half Adder -> Structural Full Adder -> Structural RCA -> Behavioral RCA) step by step. They do NOT fulfill the 'behavioral full adder + hierarchical top module' combination required by the exercise; they only serve as practice for the lecture examples.

| File | Module | Role |
|---|---|---|
| `Half_Adder.sv` | `half_adder` | Half adder (XOR + AND) |
| `Full_Adder.sv` | `full_adder` | STRUCTURAL full adder - built from two half_adder instances |
| `Ripple_Carry_Adder(Structural).sv` | `ripple_carry_adder` | Structural RCA - four hand-wired full_adder instances |
| `Ripple_Carry_Adder(Behavioral).sv` | `ripple_carry_adder_behavioral` | Fully behavioral RCA in a single module; not hierarchical, no FA sub-module (direct assigns inside a generate loop) |

*Note: these files are NOT submitted for grading; they are kept in the repo only as learning material.*

## Exercise 1 / Submission

The task requires: a behavioral description for the full adder PLUS a hierarchical 4-bit RCA that instantiates it PLUS a testbench. The three files below cover exactly that combination; this is the content handed in.

| File | Module | Role |
|---|---|---|
| `Full_Adder(Bevavioral)_Ex1.sv` | `full_adder_beh` | Behavioral full adder: SUM = A^B^CIN, COUT = (A&B)\|(CIN&(A^B)) |
| `4_bit_Ripple_Carry.sv` | `ripple_carry_4bit` | 4-bit RCA - instantiates full_adder_beh four times through a generate loop (hierarchical) |
| `tb_Ripple_Carry_4bit.sv` | `tb_Ripple_Carry_4bit` | Testbench - DUT is ripple_carry_4bit; applies 4 input vectors |

### Why this is the submission set

- The task statement literally asks for a 4-bit ripple carry adder built on a behavioral full adder.
- Full_Adder(Bevavioral)_Ex1.sv provides the behavioral FA (SUM and COUT via direct Boolean assigns).

- 4_bit_Ripple_Carry.sv instantiates that behavioral FA to build the hierarchical top-level RCA (meeting the hierarchical-project goal).
- tb_Ripple_Carry_4bit.sv provides the DUT instantiation and input vectors needed for simulation.

# 2. Submission - Source Code

## 2.1 Full_Adder(Bevavioral)_Ex1.sv

Behavioral full adder. Sum and carry out are expressed directly as Boolean assignments; there is no sub-module.

```
module full_adder_beh
(
 input logic A,
 input logic B,
 input logic CIN,

 output logic SUM,
 output logic COUT
);


assign SUM = A^B^CIN;
assign COUT = (A&B) | (CIN & (A^B));

endmodule
```

## 2.2 4_bit_Ripple_Carry.sv

4-bit ripple carry adder. The behavioral full adder (full_adder_beh) is instantiated four times through a generate / for construct. The INT net forms the carry chain (INT[0] = CIN, COUT = INT[4]).

```
module ripple_carry_4bit
(
 input logic [3:0] A,B,
 input logic CIN,

 output logic [3:0] SUM,
 output logic COUT

);

 logic [4:0] INT;
 genvar i;

 assign INT[0] = CIN;

 generate
  for(i=0;i<4;i++)
   begin
    full_adder_beh fab_0_i (.A(A[i]), .B(B[i]), .CIN(INT[i]), .SUM(SUM[i]), .COUT(INT[i+1]));
   end
 endgenerate
 assign COUT = INT[4];
endmodule
```

## 2.3 tb_Ripple_Carry_4bit.sv

Testbench. ripple_carry_4bit is instantiated as the DUT and 4 different input combinations are driven (zero inputs, carry-in with sum, plain sum, overflow case).

```
`timescale 1ns/1ps
module tb_Ripple_Carry_4bit();
 logic [3:0] A;
 logic [3:0] B;
 logic [3:0] SUM;
 logic CIN, COUT;
 ripple_carry_4bit DUT(.A(A), .B(B), .CIN(CIN), .SUM(SUM), .COUT(COUT));
 initial
  begin
   A = 0;
   B = 0;
   CIN = 0;
   #2;
   A = 5;
   B = 2;
   CIN = 1;
   #2;
   A = 4;
   B = 3;
   CIN = 0;
   #2;
   A = 15;
   B = 1;
   CIN = 0;
   #2;
   $stop;
 end
endmodule
```

# 3. Test Scenario and Expected Results

Four vectors are applied by the testbench; expected outputs for the 4-bit RCA, in decimal and as the 5-bit concatenation {COUT, SUM}:

| A | B | CIN | A + B + CIN | {COUT, SUM} (expected) |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 + 0 + 0 = 0 | COUT=0, SUM=0000 |
| 5 | 2 | 1 | 5 + 2 + 1 = 8 | COUT=0, SUM=1000 |
| 4 | 3 | 0 | 4 + 3 + 0 = 7 | COUT=0, SUM=0111 |
| 15 | 1 | 0 | 15 + 1 + 0 = 16 | COUT=1, SUM=0000 |

### ModelSim Flow

- Create a project (File -> New -> Project), name: Lab_of_Digital.
- Add the three source files to the project: Full_Adder(Bevavioral)_Ex1.sv, 4_bit_Ripple_Carry.sv, tb_Ripple_Carry_4bit.sv.
- Compile All.
- Simulate -> Start Simulation; pick tb_Ripple_Carry_4bit as the top-level unit.
- Add Wave -> All items in region to populate the waveform window.
- run -all; verify that the waveform matches the expected values above.

## 4. Conclusion

The four files under 'Exercise 1 / Practice' (Half_Adder.sv, Full_Adder.sv, Ripple_Carry_Adder(Structural).sv, Ripple_Carry_Adder(Behavioral).sv) are warm-up files reproducing the lecture examples and are not part of the formal submission.

The three files under 'Exercise 1 / Submission' fulfill the task statement (behavioral full adder + hierarchical 4-bit RCA + testbench) and are the ones handed in:

- Full_Adder(Bevavioral)_Ex1.sv (behavioral full adder - leaf module)
- 4_bit_Ripple_Carry.sv (hierarchical 4-bit ripple carry adder - top module)
- tb_Ripple_Carry_4bit.sv (testbench)